

ST-QUEST

A Field-Accurate Structured Text Course

Allen-Bradley (Studio 5000) & Siemens (TIA Portal SCL)

Published by **Oasis Games LLC**

Printable companion to the ST-Quest CLI game. Fifteen lessons with real field wiring, XIC/XIO contact reading from lesson one, pre-scaffolded code, a live PLC scan-cycle model, and a two-vendor syntax reference — all grounded in actual Rockwell and Siemens practice.

Rev 2026-07-04 · © 2026 Oasis Games LLC. All rights reserved.

How to use this course

Run the game:

```
cd C:\Dev\PLC\st-quest
python st_quest.py      (or double-click play.bat)
```

A short **how-to-play tutorial shows on every launch** (press **H** at the menu to see it again). Pick a lesson number and read the concept plus the Allen-Bradley / Siemens notes.

Every lesson file is **pre-scaffolded**: the whole working program is already written — the VAR block (each device tagged **(N.O.)/(N.C.)** with its XIC/XIO reading) and the structure — and you fill only the one blank **YOUR LOGIC** region. The file opens in Notepad automatically; write your answer, **File > Save**, come back to the terminal, and type **run**. Prefer to stay in the terminal? Type **edit** for the built-in editor.

When you run, each check reports PASS/FAIL with a hint, then your code runs live through the **PLC scan cycle** (READ INPUTS → SOLVE LOGIC → WRITE OUTPUTS) and lights up an I/O panel. Type **diff** to see exactly the line(s) you added.

In-lesson commands: `run · sim · edit · notepad · hint · diff · solution · ref · reset · brief · menu · quit`. Requires Python 3.10+ (standard library only). This handbook mirrors the game for study away from the screen.

Reading field contacts — XIC / XIO / N.O. / N.C.

Before any logic, know **how the device is wired** and **how you read it**. This is the single biggest source of real-world bugs.

Instruction	Meaning	In Structured Text	TRUE when
XIC	Examine If Closed	use the tag as-is → Tag	input bit = 1
XIO	Examine If Open	use NOT Tag	input bit = 0

N.O. (Normally Open) — open at rest, closes when actuated (start PBs, photo-eyes). Read the active state with XIC (as-is). **N.C. (Normally Closed)** — closed at rest, opens when actuated or when the wire breaks (Stop, E-stop, overloads, guards). These are **TRUE when healthy**, so you also read them with XIC and AND them into the rung as-is — never invert them. A cut wire de-energizes the input and safely stops the machine.

The fail-safe rule (memorize this)

```
Motor_Run := (Start_PB OR Motor_Run) AND Stop_PB AND Motor_OL;  (* correct *)
Motor_Run := (Start_PB OR Motor_Run) AND NOT Stop_PB;             (* WRONG - inverts a
                                                                    fail-safe N.C. device *)
```

Reach for XIO / NOT only when you truly need the inverted state — e.g. an edge-detect memory bit (`... AND NOT Sensor_Prev`) or a signal that is TRUE when *bad* (`Run := ... AND NOT Fault`).

Lesson 01 — Variables & Real I/O Declaration

Concept

Every tag you use must be declared with a **type** in a VAR ... END_VAR block:

- **BOOL** — one bit, TRUE/FALSE (a coil, a contact, a lamp).
- **INT / DINT** — whole numbers (counts, steps). AB defaults to 32-bit DINT; a Siemens INT is 16-bit.
- **REAL** — decimals (an analog level, a temperature).

But real controls care about more than the type — they care **how each field device is wired**, because that decides how you read it in code.

N.O. vs N.C. — the contact type

A pushbutton or switch is either **Normally Open (N.O.)** or **Normally Closed (N.C.)**:

- **N.O.** — open at rest, closes when actuated. Its input bit is FALSE at rest, TRUE when actuated. Typical: start buttons, photo-eyes.
- **N.C.** — closed at rest, opens when actuated (or when the wire breaks). Its input bit is TRUE at rest, FALSE when actuated. Typical fail-safe devices: stop, E-stop, overloads, guards.

XIC vs XIO — how you read it

Two instructions describe how you examine a bit (Allen-Bradley names, but the idea is universal):

- **XIC** = Examine If Closed → in Structured Text you use the tag as-is (Tag). TRUE when the bit is 1.
- **XIO** = Examine If Open → in Structured Text you use NOT Tag. TRUE when the bit is 0.

Contact type is about the **device**; XIC/XIO is about how you **read** it. The key move: safety devices are wired **N.C. and fail-safe** — energized (bit = 1) when healthy — so you read them with **XIC** and AND them into the rung as-is. A broken wire de-energizes the input and safely stops the machine. Reach for **XIO** (NOT) only when you truly need the inverted state — an edge-detect memory bit, or a signal that is TRUE when bad.

From here on, every tag is labeled **(N.O.)** or **(N.C.)** with its XIC/XIO reading, so the distinction becomes second nature.

Allen-Bradley (Studio 5000)

In Studio 5000 the default integer is DINT (32-bit) -- the Logix processor is 32-bit native. Discrete field inputs come in as BOOL tags aliased to input module points; the contact type (N.O./N.C.) is a wiring fact you record in the tag description.

Siemens (TIA Portal / SCL)

In TIA Portal / SCL, INT is a true 16-bit type (-32768..+32767) and DINT is 32-bit. Field I/O is declared in the block interface (VAR / VAR_INPUT / VAR_OUTPUT) and mapped to PLC tags in the tag table.

Your task

Declare a VAR ... END_VAR block with these six tags for a motor station, using the correct types:

```
Start_PB   : BOOL      (N.O. start pushbutton)
Stop_PB    : BOOL      (N.C. stop pushbutton)
Motor_OL   : BOOL      (N.C. overload aux contact)
Motor_Run  : BOOL      (output to contactor)
Part_Count : DINT := 0  (initialize to 0 with := )
```

```
Tank_Level : REAL
```

Add a short comment on at least one line noting its contact type.

Starter file (you fill the blank)

```
(* Lesson 01 -- Variables & Real I/O Declaration *)
(* Declare each field device with its TYPE and note its CONTACT TYPE. *)
(* Wiring key: N.O. contact -> read AS-IS (XIC) | N.C. contact -> AS-IS, fail-safe (XIC) | NOT x = X *)
VAR
  (* TODO: declare each tag with the right type (BOOL / DINT / REAL): *)
  (* Start_PB (N.O. start pushbutton) *)
  (* Stop_PB (N.C. stop pushbutton) *)
  (* Motor_OL (N.C. overload aux) *)
  (* Motor_Run (contactor output) *)
  (* Part_Count (production counter, DINT, start at 0) *)
  (* Tank_Level (analog level 0.0-100.0 %, REAL) *)
END_VAR
```

Reference solution

```
(* Lesson 01 -- Variables & Real I/O Declaration *)
(* Wiring key: N.O. contact -> read AS-IS (XIC) | N.C. contact -> AS-IS, fail-safe (XIC) | NOT x = X *)
VAR
  Start_PB : BOOL; (* N.O. start pushbutton -> XIC (as-is): TRUE when pressed *)
  Stop_PB : BOOL; (* N.C. stop pushbutton -> XIC (as-is): TRUE when NOT pressed *)
  Motor_OL : BOOL; (* N.C. overload aux -> XIC (as-is): TRUE when healthy *)
  Motor_Run : BOOL; (* output to the motor contactor *)
  Part_Count : DINT := 0; (* production counter *)
  Tank_Level : REAL; (* analog level 0.0-100.0 % *)
END_VAR
```

References: Rockwell Logix 5000 ST Programming manual (1756-PM007); Siemens SCL data types:

<https://plcblog.in/plc/siemens-tia-portal/siemens-scl-data-types-and-uses-and-value-assignments.php>; NO vs NC contacts:

<https://electrimancer.com/learn/normally-open-vs-normally-closed-contacts>

Lesson 02 — The Motor Seal-In (Real Field Wiring)

Concept

Assign vs compare

Two symbols people mix up: `:=` **assigns** a value; `=` **compares** two values. Boolean operators are AND, OR, NOT, XOR.

Reading the field devices (XIC / XIO)

A standard 3-wire motor starter wires its devices a specific way, and that wiring drives your code:

DEVICE	CONTACT	INPUT BIT READS...
Start_PB	N.O.	TRUE only while pressed
Stop_PB	N.C.	TRUE when NOT pressed, FALSE when pressed
Motor_OL	N.C.	TRUE when healthy, FALSE when tripped

Read each device the right way — **XIC** means "use the tag as-is"; **XIO** means "use NOT tag":

- **Start_PB** is (**N.O.**) → **XIC** (as-is). TRUE only while pressed — perfect to start.
- **Stop_PB** is (**N.C.**), wired fail-safe → **XIC** (as-is). It is TRUE while NOT pressed, so ANDing it in permits running.
- **Motor_OL** is (**N.C.**) → **XIC** (as-is). TRUE while the overload is healthy.

Because Stop and the overload are **normally closed**, their bits are already TRUE while running normally. You AND them in **exactly as they are — never inverted**:

```
Motor_Run := (Start_PB OR Motor_Run) AND Stop_PB AND Motor_OL;
```

Why no NOT on Stop?

Pressing the N.C. stop **opens** the circuit → Stop_PB goes FALSE → the whole expression goes FALSE → the motor drops out. Break the wire and the same thing happens. That is the **fail-safe**: the hardware provides the inversion so your logic does not. Writing NOT Stop_PB defeats it — the #1 rookie mistake. In ladder terms you examine a N.C. stop with an **XIC** contact, not XIO.

The seal-in

The (Start_PB OR Motor_Run) part is the **seal-in**: once running, Motor_Run holds itself on, so you can release the momentary start button and the motor keeps running until Stop (or the overload) breaks the rung.

Allen-Bradley (Studio 5000)

This is the ST version of a seal-in rung. In Studio 5000 ladder you'd examine Stop_PB and Motor_OL with NORMAL (XIC) contacts -- NOT XIO -- precisely because the N.C. field wiring makes the bit TRUE when safe. Using XIO (or NOT in ST) on a N.C. stop is the classic mistake that defeats the fail-safe.

Siemens (TIA Portal / SCL)

Identical in SCL: `:=`, AND, OR are standard IEC operators, and a N.C. stop is ANDed straight in. Siemens field points live in the PLC tag table with the same N.O./N.C. wiring reality behind them.

Your task

Given BOOL tags Start_PB (N.O.), Stop_PB (N.C.), Motor_OL (N.C.), and output Motor_Run, write ONE assignment implementing a real 3-wire seal-in:

- Motor_Run latches on when Start_PB is pressed and holds itself on
- Pressing Stop_PB (N.C.) drops it out
- An overload trip (Motor_OL goes FALSE) also drops it out

Stop_PB and Motor_OL are normally-closed -- AND them in as-is, do NOT use NOT.

Starter file (you fill the blank)

```
(* Lesson 02 -- The Motor Seal-In (real field wiring) *)
(* Wiring key:  N.O. contact -> read AS-IS (XIC)   |   N.C. contact -> AS-IS, fail-safe (XIC)   |   NOT x = X

VAR
    Start_PB  : BOOL;    (* N.O. start PB    -> XIC (as-is): TRUE when pressed          *)
    Stop_PB   : BOOL;    (* N.C. stop PB     -> XIC (as-is): TRUE when NOT pressed (fail-safe) *)
    Motor_OL  : BOOL;    (* N.C. overload   -> XIC (as-is): TRUE when healthy           *)
    Motor_Run : BOOL;    (* motor contactor coil (output)                          *)
END_VAR

(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
(* TODO: seal-in so Motor_Run holds itself on; AND the N.C. devices in AS-IS (XIC), never inverted. *)
(* ----- *)
```

Reference solution

```
(* Lesson 02 -- The Motor Seal-In (real field wiring) *)
(* Wiring key:  N.O. contact -> read AS-IS (XIC)   |   N.C. contact -> AS-IS, fail-safe (XIC)   |   NOT x = X

VAR
    Start_PB  : BOOL;    (* N.O. start PB    -> XIC (as-is): TRUE when pressed          *)
    Stop_PB   : BOOL;    (* N.C. stop PB     -> XIC (as-is): TRUE when NOT pressed (fail-safe) *)
    Motor_OL  : BOOL;    (* N.C. overload   -> XIC (as-is): TRUE when healthy           *)
    Motor_Run : BOOL;    (* motor contactor coil (output)                          *)
END_VAR

(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
Motor_Run := (Start_PB OR Motor_Run) AND Stop_PB AND Motor_OL;
(* ----- *)
```

References: PLC motor starter seal circuit, hardwired vs programmed:

<https://industrialmonitordirect.com/blogs/knowledgebase/plc-motor-starter-seal-circuit-hardwired-vs-programmed>; Basic PLC program for a 3-phase motor (N.O. start / N.C. stop):

<https://electrical-engineering-portal.com/basic-plc-program-for-control-of-a-three-phase-ac-motor>; Motor start/stop ladder logic tutorial: <https://plcprogramming.io/blog/motor-start-stop-ladder-logic-tutorial>

Lesson 03 — IF / ELSIF -- Tank Pump with Real Analog Input

Concept

Conditional logic uses IF / ELSIF / ELSE / END_IF. Watch the spelling: ELSIF (one E), END_IF, and a semicolon after END_IF;.

```
IF condition THEN
  (* statements *)
ELSIF other_condition THEN
  (* statements *)
END_IF;
```

Comparisons: = <> < > <= >= ('not equal' is <>).

Real-world context: Tank_Level is a REAL from a level transmitter (e.g. a 4-20 mA submersible or radar sensor) scaled to 0.0-100.0 % in the analog input module. Two setpoints (fill below 20 %, stop above 80 %) create a hysteresis band so the pump motor doesn't short-cycle -- rapid start/stop that burns out contactors and motors.

Allen-Bradley (Studio 5000)

In Studio 5000 the raw analog count is scaled (module engineering units or an SCL/CPT instruction) to give Tank_Level as a REAL. The IF/ELSIF then drives Pump_Run. In practice you'd also AND in a Pump_OL overload and permissives before energizing the pump output.

Siemens (TIA Portal / SCL)

SCL is identical: IF / ELSIF / ELSE / END_IF;. The analog channel is scaled (NORM_X + SCALE_X) into a REAL level tag, then compared to the setpoints.

Your task

Given Tank_Level : REAL (0-100 %) from the level transmitter and Pump_Run : BOOL (pump contactor output), write an IF statement that:

- Turns the pump ON (Pump_Run := TRUE) when Tank_Level < 20.0
- Turns the pump OFF (Pump_Run := FALSE) when Tank_Level > 80.0
- Leaves it unchanged between 20 and 80 (the hysteresis band)

Use IF ... ELSIF ... END_IF; -- no ELSE needed.

Starter file (you fill the blank)

```
(* Lesson 03 -- IF / ELSIF: Tank Pump from a Real Analog Level *)

VAR
  Tank_Level : REAL;    (* 0.0-100.0 % from the level transmitter (analog input) *)
  Pump_Run   : BOOL;    (* pump contactor (output) *)
END_VAR

(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
(* TODO: pump ON below 20.0 %, OFF above 80.0 %, hold its state in between (hysteresis). *)
(* ----- *)
```

Reference solution

```
(* Lesson 03 -- IF / ELSIF: Tank Pump from a Real Analog Level *)

VAR
```

```
Tank_Level : REAL;    (* 0.0-100.0 % from the level transmitter (analog input) *)
Pump_Run   : BOOL;    (* pump contactor (output) *)
END_VAR

(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
IF Tank_Level < 20.0 THEN
    Pump_Run := TRUE;
ELSIF Tank_Level > 80.0 THEN
    Pump_Run := FALSE;
END_IF;
(* ----- *)
```

References: Studio 5000 ST constructs (IF/ELSIF/END_IF):

<https://onlineplcsupport.com/structured-text-rslogix-5000-constructs/>; Siemens SCL programming guide:

<https://plcblog.in/plc/siemens-tia-portal/scl-programming-siemens-tia-portal-guide.php>

Lesson 04 — On-Delay Timers -- Where AB and Siemens DIVERGE

Concept

Timers are the #1 place Allen-Bradley and Siemens Structured Text look different -- a favorite interview topic, so learn both.

An ON-DELAY timer waits a set time after its input goes TRUE before its output turns on. Real example: after the operator presses Run (Run_Cmd), delay 5 seconds -- warning horn / lube prelube -- then energize Motor_Run.

You'll implement the 5-second on-delay in EITHER vendor's style; the checker accepts both.

Allen-Bradley (Studio 5000)

IMPORTANT: Studio 5000 ST does NOT support the plain TON/TOF/RTO you know from ladder. In ST you use the RESET variants -- TONR, TOFR, RTOR -- and the timer tag must be type FBD_TIMER (not TIMER). Preset .PRE and accumulator .ACC are in MILLISECONDS.

```
Start_Dly.PRE := 5000;           // 5 s, in ms
Start_Dly.TimerEnable := Run_Cmd;
TONR(Start_Dly);
Motor_Run := Start_Dly.DN;      // .DN = done bit
```

Siemens (TIA Portal / SCL)

TIA Portal / SCL uses the IEC timers TON, TOF, TP as function blocks, each needing an instance data block. Presets use TIME literals (T#5s, T#500ms).

```
"IEC_Timer_0_DB".TON(IN := Run_Cmd,
                     PT := T#5s,      // preset time
                     Q => Motor_Run); // output
```

Your task

Energize Motor_Run 5 seconds after Run_Cmd turns on. Write a 5-second on-delay in EITHER style:

```
Allen-Bradley:  TONR(...) with a 5000 ms preset, then drive Motor_Run from
                 the timer's .DN bit.
Siemens SCL:    TON(...) with PT := T#5s and send Q => Motor_Run.
```

The checker accepts either vendor's syntax.

Starter file (you fill the blank)

```
(* Lesson 04 -- On-Delay Timers: where Allen-Bradley and Siemens DIVERGE *)

VAR
  Run_Cmd   : BOOL;   (* N.O. run command -> XIC (as-is)   *)
  Motor_Run : BOOL;   (* output: follows Run_Cmd by 5 s   *)
  Start_Dly : TONR;   (* AB on-delay timer instance   *)
  (* Siemens equivalent lives in an IEC TON instance DB (IEC_Timer_0_DB) *)
END_VAR

(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
(* TODO: 5-second on-delay so Motor_Run turns on 5 s after Run_Cmd.
    Pick ONE vendor style below (Allen-Bradley TONR or Siemens TON). *)
(* ----- *)
```

Reference solution

```
(* Lesson 04 -- On-Delay Timers: where Allen-Bradley and Siemens DIVERGE *)

VAR
  Run_Cmd    : BOOL;    (* N.O. run command -> XIC (as-is)    *)
  Motor_Run  : BOOL;    (* output: follows Run_Cmd by 5 s    *)
  Start_Dly  : TONR;    (* AB on-delay timer instance    *)
  (* Siemens equivalent lives in an IEC TON instance DB (IEC_Timer_0_DB) *)
END_VAR

(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
(* --- Allen-Bradley (Studio 5000) --- *)
Start_Dly.PRE := 5000;           // 5 seconds, in ms
Start_Dly.TimerEnable := Run_Cmd;
TONR(Start_Dly);
Motor_Run := Start_Dly.DN;

(* --- Siemens (TIA Portal / SCL) --- *)
"IEC_Timer_0_DB".TON(IN := Run_Cmd,
                    PT := T#5s,
                    Q  => Motor_Run);

(* ----- *)
```

References: Rockwell: Studio 5000 timer & counter instructions (TONR/TOFR/RTOR): <https://www.rockwellautomation.com/en-ca/docs/studio-5000-logix-designer/37-00/contents-ditamap/instruction-set/timer-and-counter-instructions.html>;
 Allen-Bradley Logix5000 ref manual, TONR in ST:
<https://www.manualslib.com/manual/1292352/Allen-Bradley-Logix5000.html?page=129>; Siemens SCL IEC timers
 TP/TON/TOF: <https://plcblog.in/plc/siemens-tia-portal/siemens-scl-iec-timer-programming-tp-ton-tof-examples.php>

Lesson 05 — CASE -- A Traffic-Light / Sequencer State Machine

Concept

When a process moves through numbered steps (a sequencer, a batch phase, a traffic light), CASE is cleaner than a stack of IF/ELSIF:

```
CASE Seq_Step OF
  1: (* step 1 *)
  2: (* step 2 *)
  ELSE (* fallback *)
END_CASE;
```

CASE tests an integer selector. Each numbered label runs its block, and you set the selector to the NEXT step inside the block so the machine advances.

Real-world note: a production sequencer gates each transition on a condition -- usually a timer .DN bit (green for 30 s) or a sensor. Here we advance every scan to focus on CASE structure; in the field you'd wrap each step with the timer from Lesson 04.

Allen-Bradley (Studio 5000)

Studio 5000 ST supports CASE ... OF ... END_CASE; with a DINT or INT selector. This is the standard way to build a step-based sequencer in Logix ST instead of a long ladder rung sequence.

Siemens (TIA Portal / SCL)

SCL uses the same CASE ... OF ... END_CASE; and also allows ranges (1..3) and comma lists (1,3,5) as labels. Here we only need single integer labels.

Your task

Build a 3-step traffic light with CASE on Seq_Step : INT. Outputs are Green_Light, Yellow_Light, Red_Light (BOOL lamp outputs):

```
Seq_Step = 1 : Green_Light := TRUE; then advance to step 2
Seq_Step = 2 : Yellow_Light := TRUE; then advance to step 3
Seq_Step = 3 : Red_Light   := TRUE; then wrap back to step 1
```

Use CASE Seq_Step OF ... END_CASE;.

Starter file (you fill the blank)

```
(* Lesson 05 -- CASE: a Traffic-Light / Sequencer State Machine *)

VAR
  Seq_Step      : INT := 1; (* current state, 1..3 (starts at 1) *)
  Green_Light   : BOOL;    (* output *)
  Yellow_Light  : BOOL;    (* output *)
  Red_Light     : BOOL;    (* output *)
END_VAR

(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
(* TODO: CASE Seq_Step OF ... 3 states that each set a light and advance, wrapping 3 -> 1. *)
(* ----- *)
```

Reference solution

```
(* Lesson 05 -- CASE: a Traffic-Light / Sequencer State Machine *)

VAR
  Seq_Step      : INT := 1;  (* current state, 1..3 (starts at 1) *)
  Green_Light   : BOOL;      (* output *)
  Yellow_Light  : BOOL;      (* output *)
  Red_Light     : BOOL;      (* output *)
END_VAR

(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
CASE Seq_Step OF
  1:
    Green_Light := TRUE;
    Seq_Step := 2;
  2:
    Yellow_Light := TRUE;
    Seq_Step := 3;
  3:
    Red_Light := TRUE;
    Seq_Step := 1;
END_CASE;
(* ----- *)
```

References: Studio 5000 ST CASE construct: <https://onlineplcsupport.com/structured-text-rslogix-5000-constructs/>;
Siemens SCL expressions & operations: https://docs.tia.siemens.cloud/r/simatic_s7_1200_manual_collection_enus_20/programming-concepts/programming-language/scl/scl-expressions-and-operations

Lesson 06 — Machine Enable & Permissives (Safety Architecture)

Concept

A machine should only run when every safety and process condition is healthy. You gate a machine-enable seal-in with those conditions ANDed in -- and, like a stop, healthy safety signals are wired NORMALLY-CLOSED, so you AND them as-is (never invert):

```
Machine_Enable := (Start_PB OR Machine_Enable)
                  AND Safety_OK AND Guard_Closed AND Air_OK;
```

Any condition going FALSE drops the machine out immediately. Because it's a seal-in, restoring the condition does NOT auto-restart -- the operator must press Start again. That deliberate restart is a safety feature.

IMPORTANT SAFETY NOTE: a standard PLC must NOT be the sole E-stop. The E-stop function belongs to a safety relay or safety PLC (Allen-Bradley GuardLogix, Siemens F-CPU) wired to rated safety devices per ISO 13850 / ISO 13849-1. Here Safety_OK is a MONITORED status coming FROM that safety system into your normal logic -- you're gating the machine enable, not implementing the E-stop.

Each permissive is read with XIC (as-is). Using XIO (NOT) on a healthy-when-TRUE safety signal would defeat the fail-safe wiring, so never invert them.

Allen-Bradley (Studio 5000)

In Studio 5000, Safety_OK typically comes from a GuardLogix safety task or a safety relay's monitored output. Your standard-task ST reads it as a BOOL and ANDs it (with process permissives) into the enable. Guard interlocks and E-stops should be electrically separate, not daisy-chained on one input.

Siemens (TIA Portal / SCL)

In TIA Portal, the safety function lives in an F-CPU / safety program (F-DBs, F-blocks). The standard program reads the passivated/monitored safe status and gates the enable with permissives -- same :=, AND, OR.

Your task

Given Start_PB (N.O.), and the healthy-when-TRUE signals Safety_OK, Guard_Closed and Air_OK, plus output Machine_Enable, write ONE seal-in assignment so:

- Machine_Enable latches on when Start_PB is pressed AND all conditions OK
- It holds after Start releases
- Any of Safety_OK / Guard_Closed / Air_OK going FALSE drops it out
- After a condition is restored it stays OFF until Start is pressed again

All three conditions are healthy-when-TRUE -- AND them in as-is, no NOT.

Starter file (you fill the blank)

```
(* Lesson 06 -- Machine Enable & Permissives (Safety Architecture) *)
(* Wiring key:  N.O. contact -> read AS-IS (XIC)   |   N.C. contact -> AS-IS, fail-safe (XIC)   |   NOT x = XIO *)

VAR
  Start_PB      : BOOL;  (* N.O. start PB      -> XIC (as-is)                *)
  Safety_OK     : BOOL;  (* from safety relay -> XIC (as-is): TRUE = safe    *)
  Guard_Closed  : BOOL;  (* N.C. guard switch -> XIC (as-is): TRUE = closed  *)
```

```

    Air_OK          : BOOL;  (* pressure switch    -> XIC (as-is): TRUE = pressure OK *)
    Machine_Enable  : BOOL;  (* output                                     *)
END_VAR

(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
(* TODO: one seal-in enable rung; AND the healthy-when-TRUE permissives in AS-IS (XIC), never inverted. *)
(* ----- *)

```

Reference solution

```

(* Lesson 06 -- Machine Enable & Permissives (Safety Architecture) *)
(* Wiring key:  N.O. contact -> read AS-IS (XIC)      |  N.C. contact -> AS-IS, fail-safe (XIC)      |  NOT x = X
VAR
    Start_PB        : BOOL;  (* N.O. start PB        -> XIC (as-is)                                *)
    Safety_OK        : BOOL;  (* from safety relay -> XIC (as-is): TRUE = safe          *)
    Guard_Closed     : BOOL;  (* N.C. guard switch -> XIC (as-is): TRUE = closed      *)
    Air_OK           : BOOL;  (* pressure switch   -> XIC (as-is): TRUE = pressure OK *)
    Machine_Enable   : BOOL;  (* output                                     *)
END_VAR

(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
Machine_Enable := (Start_PB OR Machine_Enable) AND Safety_OK AND Guard_Closed AND Air_OK;
(* ----- *)

```

References: E-stop safety circuit in PLC code (safety relay / safety PLC, not standard logic):

<https://plcprogramming.io/blog/e-stop-safety-circuit-plc-ladder-logic/>; ISO 13850 emergency stop requirements:

<https://machinerysafety101.com/2026/05/18/iso-13850-emergency-stop-requirements/>; E-stop + safety relay wiring (N.C. devices, interlocks): https://www.dadisick.com/blog/Wiring-Diagram-of-Emergency-Stop-Button-and-Safety-Relay-How-to-Properly-Connect-for-Safety_b4955

Lesson 07 — FOR Loops & Arrays -- Scanning an I/O Bank

Concept

A FOR loop repeats a block a fixed number of times; arrays hold many values under one name. Together they let you process a bank of I/O in a few lines:

```
FOR i := 0 TO 7 DO
    Total := Total + Data[i];
END_FOR;
```

Syntax: FOR i := start TO end [BY step] DO ... END_FOR;. The whole loop runs inside ONE PLC scan, so keep counts sane -- a huge loop lengthens scan time.

Array indexing is Data[i]. In Studio 5000 arrays are 0-based (a DINT[8] has indices 0..7), which is what we use here.

Allen-Bradley (Studio 5000)

Studio 5000: arrays are 0-based, e.g. Data : DINT[8] -> Data[0]..Data[7]. FOR loops are common for packing/unpacking status bits or scaling I/O banks. Watch scan time on large iteration counts.

Siemens (TIA Portal / SCL)

SCL arrays can have arbitrary bounds, e.g. ARRAY[1..8] OF INT, so you'd write FOR i := 1 TO 8. Same FOR ... END_FOR structure; just mind the index base. For this exercise use 0..7 to match the data given.

Your task

Given Data : an array of 8 integers (indices 0..7), compute two outputs with a single FOR loop over i := 0 TO 7:

- Total : the sum of all Data[i]
- Alarm_Count : how many Data[i] are greater than 100

Initialise Total and Alarm_Count to 0 before the loop.

Starter file (you fill the blank)

```
(* Lesson 07 -- FOR Loops & Arrays: Scanning an I/O Bank *)

VAR
    Data      : ARRAY[0..7] OF INT;  (* 8-channel input bank, indices 0..7 *)
    i         : INT;                 (* loop index *)
    Total     : DINT;                (* output: running sum *)
    Alarm_Count : DINT;              (* output: count of channels > 100 *)
END_VAR

(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
(* TODO: FOR i := 0 TO 7 -- sum Data[i] into Total and count Data[i] > 100 into Alarm_Count. *)
(* ----- *)
```

Reference solution

```
(* Lesson 07 -- FOR Loops & Arrays: Scanning an I/O Bank *)

VAR
    Data      : ARRAY[0..7] OF INT;  (* 8-channel input bank, indices 0..7 *)
    i         : INT;                 (* loop index *)
    Total     : DINT;                (* output: running sum *)
    Alarm_Count : DINT;              (* output: count of channels > 100 *)
```

END_VAR

```
(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
Total := 0;
Alarm_Count := 0;
FOR i := 0 TO 7 DO
    Total := Total + Data[i];
    IF Data[i] > 100 THEN
        Alarm_Count := Alarm_Count + 1;
    END_IF;
END_FOR;
(* ----- *)
```

References: Studio 5000 ST loops (FOR/WHILE): <https://onlineplcsupport.com/structured-text-rslogix-5000-constructs/>;
Siemens SCL FOR loop & arrays: <https://plcblog.in/plc/siemens-tia-portal/scl-programming-siemens-tia-portal-guide.php>

Lesson 08 — Functions & Function Blocks (Reusable Logic)

Concept

Reusable logic keeps programs clean. IEC / Siemens SCL text uses two forms:

```
FUNCTION (FC)      -- stateless; returns a value; no memory between calls
FUNCTION_BLOCK (FB) -- has memory (an instance); e.g. a timer, a motor FB
```

A FUNCTION assigns its result to its own name and closes with END_FUNCTION:

```
FUNCTION ClampHigh : REAL
VAR_INPUT
    IN : REAL;
    HI : REAL;
END_VAR
    IF IN > HI THEN
        ClampHigh := HI;
    ELSE
        ClampHigh := IN;
    END_IF;
END_FUNCTION
```

Allen-Bradley (Studio 5000)

Studio 5000 does NOT type FUNCTION/END_FUNCTION as text. Rockwell packages reusable logic as an Add-On Instruction (AOI), built in the IDE with defined parameters and local tags, then called like MyAOI(params). Mapping: an AB AOI is roughly a Siemens FB (it stores state in its own tag); a plain subroutine is like a Siemens FC. So the text below is the Siemens/IEC form -- learn it, and know AB's equivalent is the AOI.

Siemens (TIA Portal / SCL)

TIA Portal SCL uses exactly this: FUNCTION (FC) for stateless returns, and FUNCTION_BLOCK (FB) which needs an instance data block and retains static data across scans. One FB can be instantiated many times (one DB per motor/axis).

Your task

Write an IEC/SCL-style FUNCTION named ClampHigh that returns a REAL. It takes two REAL inputs, IN and HI, and returns IN limited to a maximum of HI:

```
- if IN > HI, the result is HI
- otherwise the result is IN
```

Declare the inputs in a VAR_INPUT ... END_VAR block, assign the result to the function's own name (ClampHigh := ...), and close with END_FUNCTION.

Starter file (you fill the blank)

```
(* Lesson 08 -- Functions & Function Blocks (Reusable Logic) *)
(* Write a FUNCTION ClampHigh : REAL that limits IN so it never exceeds HI.  *)
(* (Pure math here -- no field contacts, so no XIC/XIO.)                      *)

FUNCTION ClampHigh : REAL
VAR_INPUT
    (* TODO: declare the two REAL inputs, IN and HI *)
END_VAR
(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
(* TODO: return IN, but limited to a maximum of HI *)
```

```
(* ----- *)
END_FUNCTION
```

Reference solution

```
(* Lesson 08 -- Functions & Function Blocks (Reusable Logic) *)
(* Write a FUNCTION ClampHigh : REAL that limits IN so it never exceeds HI. *)
(* (Pure math here -- no field contacts, so no XIC/XIO.) *)

FUNCTION ClampHigh : REAL
VAR_INPUT
    IN : REAL;
    HI : REAL;
END_VAR
(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
    IF IN > HI THEN
        ClampHigh := HI;
    ELSE
        ClampHigh := IN;
    END_IF;
(* ----- *)
END_FUNCTION
```

References: Siemens vs Allen-Bradley function blocks (AOI vs FB/FC):

<https://www.dmcinfo.com/blog/29940/siemens-vs-allen-bradley-function-blocks/>; Structuring PLC apps: Rockwell vs Siemens TIA Portal:

<https://www.plcskilltree.com/blog/structuring-plc-applications-rockwell-studio-5000-vs-siemens-tia-portal/>; Siemens SCL FC/FB reusable code:

<https://industrialmonitordirect.com/blogs/knowledgebase/siemens-step-7-fb-fc-functions-reusable-conveyor-control-code>

Lesson 09 — Counters & Rising-Edge Detection

Concept

Machines count things: parts made, cycles run, bottles filled. The trick is the EDGE — you want to count once per event, not every scan the sensor is TRUE.

A rising-edge "one-shot" uses a memory bit that remembers last scan's value:

```
IF Part_Sensor AND NOT Sensor_Prev THEN
    Part_Count := Part_Count + 1;    (* fires once, on the 0->1 edge *)
END_IF;
Sensor_Prev := Part_Sensor;        (* remember for next scan *)
```

Without the edge test, Part_Count would race up every scan the sensor is held on. The IEC counters (CTU/CTD/CTUD) wrap this pattern in a function block.

Notice the two readings working together: Part_Sensor is used as-is (XIC) while the memory bit is inverted, NOT Sensor_Prev (XIO). A rising edge is simply XIC(input) AND XIO(memory).

Allen-Bradley (Studio 5000)

Studio 5000 has a one-shot (ONS) and the CTU counter. In ST you often just do the manual edge shown above with a BOOL memory tag, or call CTU with a COUNTER/FBD_COUNTER structure: check .DN when .ACC >= .PRE.

Siemens (TIA Portal / SCL)

TIA Portal / SCL uses the IEC counter CTU as a function block with an instance DB:

"IEC_Counter_0_DB".CTU(CU := Part_Sensor, PV := 10, Q => Full, CV => Count); CU is edge-triggered internally, so the FB does the one-shot for you.

Your task

Count parts on the RISING edge of Part_Sensor (BOOL). Given the output Part_Count : DINT and a memory bit Sensor_Prev : BOOL, increment Part_Count by 1 each time Part_Sensor goes from FALSE to TRUE — and only then.

Remember to update Sensor_Prev at the end so the edge is detected next scan.

Starter file (you fill the blank)

```
(* Lesson 09 -- Counters & Rising-Edge Detection *)
(* Edge = input TRUE now AND memory FALSE last scan -> XIC(input) AND XIO(memory). *)

VAR
    Part_Sensor : BOOL;    (* N.O. photo-eye -> XIC (as-is): TRUE while a part is present *)
    Sensor_Prev : BOOL;    (* last scan's sensor state (edge-detect memory) *)
    Part_Count  : DINT;    (* output: parts counted *)
END_VAR

(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
(* TODO: +1 once per rising (0->1) edge of Part_Sensor, then remember this scan in Sensor_Prev. *)
(* ----- *)
```

Reference solution

```
(* Lesson 09 -- Counters & Rising-Edge Detection *)
(* Edge = input TRUE now AND memory FALSE last scan -> XIC(input) AND XIO(memory). *)
```

```
VAR
    Part_Sensor : BOOL;    (* N.O. photo-eye -> XIC (as-is): TRUE while a part is present *)
    Sensor_Prev : BOOL;    (* last scan's sensor state (edge-detect memory) *)
    Part_Count  : DINT;    (* output: parts counted *)
END_VAR

(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
IF Part_Sensor AND NOT Sensor_Prev THEN
    Part_Count := Part_Count + 1;
END_IF;
Sensor_Prev := Part_Sensor;
(* ----- *)
```

References: Rockwell Studio 5000 timer & counter instructions (CTU/ONS): <https://www.rockwellautomation.com/en-ca/docs/studio-5000-logix-designer/37-00/contents-ditomap/instruction-set/timer-and-counter-instructions.html>; Siemens SCL IEC counters (CTU/CTD/CTUD): <https://plcblog.in/plc/siemens-tia-portal/siemens-scl-iec-counter-programming.php>

Lesson 10 — Up-Down Counter (CTUD)

Concept

Sometimes you count both ways: parts IN vs parts OUT of a buffer, cars entering and leaving a lot, bottles added and removed. That's an up-down counter (CTUD).

Do it with a rising-edge one-shot on each input (see Lesson 9):

```
IF In_Sensor AND NOT In_Prev THEN Count := Count + 1; END_IF;
IF Out_Sensor AND NOT Out_Prev THEN Count := Count - 1; END_IF;
In_Prev := In_Sensor; Out_Prev := Out_Sensor;
```

Count now tracks how many are currently in the buffer.

Allen-Bradley (Studio 5000)

Studio 5000 has CTUD (up/down) with .ACC; or do the manual edges above with BOOL memory tags.

Siemens (TIA Portal / SCL)

SCL uses the IEC CTUD FB: CU (count up), CD (count down), with CV giving the current value; both inputs are edge-triggered internally.

Your task

A buffer counts items IN and OUT. Given In_Sensor and Out_Sensor (BOOL) with memory bits In_Prev, Out_Prev, and output Count : DINT:

- +1 on the rising edge of In_Sensor
- -1 on the rising edge of Out_Sensor

Update both memory bits at the end.

Starter file (you fill the blank)

```
(* Lesson 10 -- Up-Down Counter (CTUD) *)

VAR
  In_Sensor   : BOOL; (* N.O. entry photo-eye -> XIC (as-is) *)
  Out_Sensor  : BOOL; (* N.O. exit photo-eye -> XIC (as-is) *)
  In_Prev     : BOOL; (* entry edge memory *)
  Out_Prev    : BOOL; (* exit edge memory *)
  Count       : DINT; (* output: items currently in buffer *)
END_VAR

(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
(* TODO: +1 on each entry rising edge, -1 on each exit rising edge; then update both memory bits. *)
(* ----- *)
```

Reference solution

```
(* Lesson 10 -- Up-Down Counter (CTUD) *)

VAR
  In_Sensor   : BOOL; (* N.O. entry photo-eye -> XIC (as-is) *)
  Out_Sensor  : BOOL; (* N.O. exit photo-eye -> XIC (as-is) *)
  In_Prev     : BOOL; (* entry edge memory *)
  Out_Prev    : BOOL; (* exit edge memory *)
  Count       : DINT; (* output: items currently in buffer *)
END_VAR
```

```
(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
IF In_Sensor AND NOT In_Prev THEN
    Count := Count + 1;
END_IF;
IF Out_Sensor AND NOT Out_Prev THEN
    Count := Count - 1;
END_IF;
In_Prev := In_Sensor;
Out_Prev := Out_Sensor;
(* ----- *)
```

References: Rockwell CTUD (timer & counter instructions): <https://www.rockwellautomation.com/en-ca/docs/studio-5000-logic-designer/37-00/contents-ditamap/instruction-set/timer-and-counter-instructions.html>; Siemens SCL IEC counters CTU/CTD/CTUD: <https://plcblog.in/plc/siemens-tia-portal/siemens-scl-iec-counter-programming.php>

Lesson 11 — Analog Scaling (Raw Counts -> Engineering Units)

Concept

An analog input arrives as RAW COUNTS, not real units. You scale it linearly to engineering units (% , PSI, degC). For a 0-10 V channel:

Siemens: 0 V = 0 counts, 10 V = 27648 counts
Allen-Bradley: often 0-32767 (or module engineering-unit scaled)

Linear scale to 0-100 %: $\text{Level} := \text{Raw} * 100.0 / 27648.0;$

General form: $\text{EU} := (\text{Raw} - \text{RawMin}) * (\text{EuMax} - \text{EuMin}) / (\text{RawMax} - \text{RawMin}) + \text{EuMin}.$

Allen-Bradley (Studio 5000)

Studio 5000: scale with the module's engineering units, or a CPT/SCL expression on the raw tag to a REAL.

Siemens (TIA Portal / SCL)

SCL uses NORM_X (raw -> 0.0..1.0) then SCALE_X (-> your EU range). The one-liner here does both in a single expression.

Your task

Given Raw : INT from a 0-10 V channel (0..27648 counts), compute Level : REAL as 0.0..100.0 %. Use a linear scale.

```
Level := Raw * 100.0 / 27648.0;
```

Starter file (you fill the blank)

```
(* Lesson 11 -- Analog Scaling: Raw Counts -> Engineering Units *)

VAR
    Raw    : INT;    (* 0..27648 raw counts from the analog input card *)
    Level  : REAL;    (* output: 0.0..100.0 engineering units (%) *)
END_VAR

(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
(* TODO: scale Raw (0..27648) linearly to Level (0.0..100.0 %). Use REAL math. *)
(* ----- *)
```

Reference solution

```
(* Lesson 11 -- Analog Scaling: Raw Counts -> Engineering Units *)

VAR
    Raw    : INT;    (* 0..27648 raw counts from the analog input card *)
    Level  : REAL;    (* output: 0.0..100.0 engineering units (%) *)
END_VAR

(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
Level := Raw * 100.0 / 27648.0;
(* ----- *)
```

References: Siemens NORM_X / SCALE_X scaling:

<https://plcblog.in/plc/siemens-tia-portal/norm-x-and-scale-x-in-siemens-tia-portal.php>; Studio 5000 analog scaling (engineering units):

<https://www.rockwellautomation.com/en-ca/docs/studio-5000-logix-designer/37-00/contents-ditamap.html>

Lesson 12 — HMI Alarms -- Latch & Acknowledge

Concept

An HMI/SCADA alarm must LATCH the moment a condition trips, so a brief fault is never missed, and stay latched until the operator ACKNOWLEDGES it AND the condition has cleared. That's a seal-in with an acknowledge:

```
IF Alarm_In THEN
    Alarm_Latched := TRUE;      (* trip and hold *)
ELSIF Ack THEN
    Alarm_Latched := FALSE;     (* clear only when condition is gone *)
END_IF;
```

Because the ELSIF only runs when Alarm_In is FALSE, pressing Ack while the fault is still active does NOT clear it -- exactly what you want.

Allen-Bradley (Studio 5000)

Studio 5000 has ALARM/ALMD instructions with built-in latch + ack; the logic above is the same idea in plain ST for any HMI tag.

Siemens (TIA Portal / SCL)

TIA Portal WinCC alarms latch in the HMI, but the PLC-side latch/ack logic is identical ST. 'First-out' annunciation extends this by remembering which alarm tripped first.

Your task

Given Alarm_In (BOOL, TRUE when abnormal) and Ack (BOOL, operator acknowledge), drive Alarm_Latched (BOOL):

- Latch TRUE as soon as Alarm_In is TRUE
- Clear to FALSE only when Ack is pressed AND Alarm_In has cleared

Use IF Alarm_In ... ELSIF Ack ... END_IF;.

Starter file (you fill the blank)

```
(* Lesson 12 -- HMI Alarms: Latch & Acknowledge *)

VAR
    Alarm_In      : BOOL;  (* fault condition      -> XIC (as-is): TRUE = fault active *)
    Ack           : BOOL;  (* N.O. acknowledge PB -> XIC (as-is)                *)
    Alarm_Latched : BOOL;  (* output: latched alarm shown on the HMI                *)
END_VAR

(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
(* TODO: latch TRUE on Alarm_In; clear on Ack ONLY after the fault has cleared. *)
(* ----- *)
```

Reference solution

```
(* Lesson 12 -- HMI Alarms: Latch & Acknowledge *)

VAR
    Alarm_In      : BOOL;  (* fault condition      -> XIC (as-is): TRUE = fault active *)
    Ack           : BOOL;  (* N.O. acknowledge PB -> XIC (as-is)                *)
    Alarm_Latched : BOOL;  (* output: latched alarm shown on the HMI                *)
END_VAR
```



```
(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
IF Alarm_In THEN
    Alarm_Latched := TRUE;
ELSIF Ack THEN
    Alarm_Latched := FALSE;
END_IF;
(* ----- *)
```

References: Studio 5000 ALMD/ALARM instructions overview:

<https://www.rockwellautomation.com/en-ca/docs/studio-5000-logix-designer/37-00/contents-ditamap.html>; HMI alarm latch/acknowledge pattern: <https://plcprogramming.io/blog/plc-alarm-management>

Lesson 13 — Modbus Mapping -- Expose Data to an HMI / App

Concept

To let an HMI, SCADA, or app read/write your PLC over Modbus TCP, you MAP tags onto Modbus objects. Four object types:

```
Coils          (0xxxx) - read/write bits   (commands)
Discrete Inputs (1xxxx) - read-only bits   (status)
Holding Registers (4xxxx) - read/write 16-bit (setpoints)
Input Registers (3xxxx) - read-only 16-bit (measurements)
```

You stage internal tags into the mapped tags each scan:

```
MB_Cmd_Run    := System_Run;   (* coil           *)
MB_Sts_Fault  := Fault;        (* discrete input *)
MB_HR_Setpt   := Setpoint;     (* holding register*)
```

Allen-Bradley (Studio 5000)

Micro800/Logix expose Modbus TCP via the Ethernet port config or MSG instructions; you assign tags to Modbus addresses in the mapping.

Siemens (TIA Portal / SCL)

S7-1200/1500 use the MB_SERVER (slave) / MB_CLIENT FBs; the HMI reads the mapped holding registers. Same idea: internal tag -> Modbus address.

Your task

Stage three internal tags onto their Modbus-mapped tags so an app can see them:

- MB_Cmd_Run := System_Run (coil / command)
- MB_Sts_Fault := Fault (discrete input / status)
- MB_HR_Setpt := Setpoint (holding register / setpoint)

Starter file (you fill the blank)

```
(* Lesson 13 -- Modbus Mapping: Expose Data to an HMI / App *)

VAR
  (* --- internal application tags --- *)
  System_Run  : BOOL;   (* running flag      *)
  Fault       : BOOL;   (* fault flag        *)
  Setpoint    : INT;    (* internal setpoint *)
  (* --- Modbus image the HMI / app reads & writes --- *)
  MB_Cmd_Run  : BOOL;   (* coil             (0x) *)
  MB_Sts_Fault : BOOL;  (* discrete input   (1x) *)
  MB_HR_Setpt : INT;    (* holding register (4x) *)
END_VAR

(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
(* TODO: stage the three internal tags onto their matching Modbus image tags. *)
(* ----- *)
```

Reference solution

```
(* Lesson 13 -- Modbus Mapping: Expose Data to an HMI / App *)
```

```
VAR
(* --- internal application tags --- *)
System_Run   : BOOL;   (* running flag      *)
Fault        : BOOL;   (* fault flag    *)
Setpoint     : INT;    (* internal setpoint *)
(* --- Modbus image the HMI / app reads & writes --- *)
MB_Cmd_Run   : BOOL;   (* coil          (0x) *)
MB_Sts_Fault : BOOL;   (* discrete input (1x) *)
MB_HR_Setpt  : INT;    (* holding register (4x) *)
END_VAR

(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
MB_Cmd_Run   := System_Run;
MB_Sts_Fault := Fault;
MB_HR_Setpt  := Setpoint;
(* ----- *)
```

References: Siemens MB_SERVER / MB_CLIENT (Modbus TCP):

<https://plcblog.in/plc/siemens-tia-portal/modbus-tcp-communication-in-siemens-tia-portal.php>; Modbus object model (coils/registers) overview: <https://plcprogramming.io/blog/modbus-protocol>

Lesson 14 — Function Blocks with Memory -- a Reusable Motor Starter

Concept

A FUNCTION returns a value and forgets everything. A FUNCTION_BLOCK keeps its own MEMORY between scans (an instance) -- so you can write control logic ONCE and drop an instance on every motor. The seal-in is the perfect example, because it reads its own previous output:

```
FUNCTION_BLOCK MotorStarter
VAR_INPUT  Start : BOOL; Stop : BOOL; OL : BOOL; END_VAR
VAR_OUTPUT Run  : BOOL; END_VAR
    Run := (Start OR Run) AND Stop AND OL;    (* Run is the FB's memory *)
END_FUNCTION_BLOCK
```

You then declare an instance and call it: M1(Start:=..., Stop:=..., OL:=...); and read M1.Run. Each instance remembers its own Run.

Allen-Bradley (Studio 5000)

Studio 5000 packages reusable logic as an Add-On Instruction (AOI); its state lives in the AOI's own tag -- exactly like an FB instance.

Siemens (TIA Portal / SCL)

TIA Portal FB needs an instance data block (one per motor); the DB holds Run across scans. That instance memory is what makes the seal-in work.

Your task

Write a FUNCTION_BLOCK named MotorStarter with: VAR_INPUT Start, Stop, OL : BOOL VAR_OUTPUT Run : BOOL Its body implements the 3-wire seal-in: Run := (Start OR Run) AND Stop AND OL; (Stop and OL are N.C. -- TRUE when safe -- so AND them in as-is.)

Starter file (you fill the blank)

```
(* Lesson 14 -- Function Blocks with Memory: a Reusable Motor Starter *)
(* Wiring key:  N.O. contact -> read AS-IS (XIC)    |    N.C. contact -> AS-IS, fail-safe (XIC)    |    NOT x = X

FUNCTION_BLOCK MotorStarter
VAR_INPUT
    Start : BOOL;    (* N.O. start    -> XIC (as-is)                *)
    Stop  : BOOL;    (* N.C. stop      -> XIC (as-is, fail-safe) *)
    OL    : BOOL;    (* N.C. overload -> XIC (as-is)                *)
END_VAR
VAR_OUTPUT
    Run : BOOL;    (* sealed-in run output (keeps its state) *)
END_VAR
(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
(* TODO: seal-in so Run holds itself on; AND Stop and OL in AS-IS (XIC). *)
(* ----- *)
END_FUNCTION_BLOCK
```

Reference solution

```
(* Lesson 14 -- Function Blocks with Memory: a Reusable Motor Starter *)
(* Wiring key:  N.O. contact -> read AS-IS (XIC)    |    N.C. contact -> AS-IS, fail-safe (XIC)    |    NOT x = X

FUNCTION_BLOCK MotorStarter
```

```
VAR_INPUT
    Start : BOOL;    (* N.O. start    -> XIC (as-is)          *)
    Stop  : BOOL;    (* N.C. stop     -> XIC (as-is, fail-safe) *)
    OL    : BOOL;    (* N.C. overload -> XIC (as-is)          *)
END_VAR
VAR_OUTPUT
    Run : BOOL;      (* sealed-in run output (keeps its state) *)
END_VAR
(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
    Run := (Start OR Run) AND Stop AND OL;
(* ----- *)
END_FUNCTION_BLOCK
```

References: Siemens vs Allen-Bradley function blocks (AOI vs FB/FC):

<https://www.dmcinfo.com/blog/29940/siemens-vs-allen-bradley-function-blocks/>; Siemens SCL FB with instance DB:

<https://plcblog.in/plc/siemens-tia-portal/scl-programming-siemens-tia-portal-guide.php>

Lesson 15 — Sequential Function Chart (SFC) -- a Batch Sequencer

Concept

An SFC is steps + transitions: each STEP runs actions, and you advance to the next step ONLY when its TRANSITION condition becomes TRUE. (Lesson 5 advanced every scan; a real sequencer gates each move.) In ST you hand-code an SFC as a guarded CASE state machine:

```
CASE Step OF
  1: Valve := TRUE;
    IF Level_Full THEN Valve := FALSE; Step := 2; END_IF;  (* transition *)
  ...
END_CASE;
```

Each step energizes its own output and moves on only when its condition is met.

Allen-Bradley (Studio 5000)

Studio 5000 has native SFC as a language; when you write it in ST, the guarded-CASE pattern is the standard hand-coded equivalent.

Siemens (TIA Portal / SCL)

Siemens GRAPH is their SFC editor; in SCL you use the same CASE + IF-transition pattern shown here.

Your task

Batch sequencer with Step : INT (0=Idle,1=Fill,2=Heat,3=Drain). Inputs: Start, Level_Full, Temp_OK, Level_Empty : BOOL. Outputs: Fill_Valve, Heater, Drain_Valve : BOOL.

0 Idle : all outputs OFF; go to 1 when Start
 1 Fill : Fill_Valve ON; go to 2 when Level_Full (turn Fill_Valve off)
 2 Heat : Heater ON; go to 3 when Temp_OK (turn Heater off)
 3 Drain : Drain_Valve ON; go to 0 when Level_Empty (turn Drain_Valve off)

Use CASE Step OF ... END_CASE; with an IF transition in each step.

Starter file (you fill the blank)

```
(* Lesson 15 -- Sequential Function Chart (SFC): a Batch Sequencer *)

VAR
  Step      : INT;    (* current SFC step, 0..3 *)
  Start     : BOOL;   (* N.O. start PB -> XIC (as-is) *)
  Level_Full : BOOL;   (* hi-level float -> XIC (as-is): TRUE=full *)
  Temp_OK   : BOOL;   (* thermostat -> XIC (as-is): TRUE=at temp*)
  Level_Empty : BOOL;  (* lo-level float -> XIC (as-is): TRUE=empty *)
  Fill_Valve : BOOL;   (* output *)
  Heater    : BOOL;   (* output *)
  Drain_Valve : BOOL;  (* output *)
END_VAR

(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
(* TODO: CASE Step OF 0..3 -- each step drives its outputs and has ONE guarded transition. *)
(* ----- *)
```

Reference solution

```

(* Lesson 15 -- Sequential Function Chart (SFC): a Batch Sequencer *)

VAR
  Step      : INT;    (* current SFC step, 0..3          *)
  Start     : BOOL;   (* N.O. start PB -> XIC (as-is)          *)
  Level_Full : BOOL;   (* hi-level float -> XIC (as-is): TRUE=full *)
  Temp_OK   : BOOL;   (* thermostat -> XIC (as-is): TRUE=at temp*)
  Level_Empty : BOOL;  (* lo-level float -> XIC (as-is): TRUE=empty *)
  Fill_Valve : BOOL;   (* output *)
  Heater    : BOOL;   (* output *)
  Drain_Valve : BOOL;  (* output *)
END_VAR

(* ----- YOUR LOGIC : write your answer between these two lines ----- *)
CASE Step OF
  0:
    Fill_Valve := FALSE; Heater := FALSE; Drain_Valve := FALSE;
    IF Start THEN Step := 1; END_IF;
  1:
    Fill_Valve := TRUE;
    IF Level_Full THEN Fill_Valve := FALSE; Step := 2; END_IF;
  2:
    Heater := TRUE;
    IF Temp_OK THEN Heater := FALSE; Step := 3; END_IF;
  3:
    Drain_Valve := TRUE;
    IF Level_Empty THEN Drain_Valve := FALSE; Step := 0; END_IF;
END_CASE;
(* ----- *)

```

References: Studio 5000 ST CASE / state machines: <https://onlineplcsupport.com/structured-text-rslogix-5000-constructs/>;
 IEC 61131-3 SFC overview: <https://plcblog.in/plc/sequential-function-chart-sfc.php>

Disclaimers

Educational use only. ST-Quest is a learning aid published by Oasis Games LLC, not a certified engineering or safety tool. Verify every example against your exact controller firmware / TIA Portal version and your site's wiring, grounding, and safety standards before using it on real equipment; you are solely responsible for safe design and operation. Any 120/240 V or mains wiring must be performed by a qualified licensed electrician per the NEC and local codes. This material is provided "as is", without warranty of any kind.

Trademarks. Allen-Bradley, Studio 5000, Logix, and Micro850 are trademarks of Rockwell Automation, Inc. Siemens, TIA Portal, SIMATIC, and SCL are trademarks of Siemens AG. Modbus is a trademark of Schneider Electric. Oasis Games LLC is an independent educational publisher and is not affiliated with, endorsed by, or sponsored by any referenced company.

Keep going

Work every lesson in the game until all 15 show a green check. Then open the free-play REPL (press **F** at the menu) and write your own Structured Text, and study `reference/ST_cheatsheet_AB_vs_Siemens.md` for the full two-vendor comparison. Validate all syntax against your exact controller firmware / TIA Portal version and your site's wiring and safety standards before deploying anything.